Aspera Orchestrator Plugin Development Guide

IBM Aspera

2026/10/21

Contents

1 Introduction **3**
- 1.1 What is a Plugin? . . . 3
- 1.2 How Plugins are Detected and Loaded . . . 3

2 Plugin Structure **4**
- 2.1 Additional Ruby Files . . . 4
- 2.2 File Naming Conventions . . . 4
- 2.3 Directory Location . . . 5

3 Plugin Architecture Patterns **6**
- 3.1 Class Structure . . . 6
- 3.2 Core Methods . . . 6
  - 3.2.1 Required Methods . . . 6
  - 3.2.2 Optional Methods (for advanced features) . . . 6
- 3.3 Execution Patterns . . . 7

4 Required Elements **8**
- 4.1 Main Ruby Class . . . 8
- 4.2 Required Methods . . . 8
  - 4.2.1 `self.version` . . . 8
  - 4.2.2 `self.display_name` (Optional but Recommended) . . . 8
  - 4.2.3 `name` (Optional) . . . 8
  - 4.2.4 `inputs_spec` . . . 9
  - 4.2.5 `outputs_spec` . . . 9
  - 4.2.6 `category` . . . 9
  - 4.2.7 `execute` . . . 10
- 4.3 `metadata.yml` File . . . 10

5 API Reference **12**
- 5.1 Data Types . . . 12
- 5.2 Execution Status . . . 12

6 Simple Example: ”Hello World” Plugin **13**
- 6.1 Directory Structure . . . 13
- 6.2 `hello_world_simple.rb` . . . 13
- 6.3 metadata.yml . . . 14
- 6.4 How It Works . . . 14

7 Testing Your Plugin **15**
- 7.1 Development vs Production Paths . . . 15
- 7.2 Testing Steps . . . 15
- 7.3 Important Notes . . . 15

8 Advanced Features **16**
- 8.1 Database Migrations . . . 16
- 8.2 Asynchronous Execution . . . 16
- 8.3 Trigger Plugins . . . 17

8.4 Lifecycle Control (Pause/Resume/Cancel/Rollback) . . . 18
8.5 Input Validation . . . 19
8.6 Helper Methods . . . 19
8.7 Dynamic Input/Output Specifications . . . 19

9 Plugin Patterns Usage Statistics **21**
9.1 Common Patterns . . . 21
9.2 Key Takeaways . . . 21

10 Best Practices **22**

11 Advanced Usage Patterns **23**
11.1 Using Payload Variables . . . 23
11.2 Remote Node Integration . . . 23
11.3 Multi-Type Conversion . . . 24

12 Plugin Development Alternatives and Possibilities **25**
12.1 Plugin Types and Use Cases . . . 25
12.2 Configuration UI Options . . . 25
12.2.1 Standard Form Fields . . . 25
12.2.2 Dynamic Configuration . . . 26
12.2.3 Help Documentation . . . 26
12.3 External Dependencies Management . . . 26
12.3.1 Using Gemfile . . . 26
12.3.2 System Dependencies . . . 26
12.4 Plugin Versioning and Upgrades . . . 26
12.4.1 Version Management . . . 26
12.4.2 Upgrade Process . . . 26
12.4.3 Deprecation . . . 27
12.5 Plugin Distribution . . . 27
12.5.1 Installation . . . 27
12.6 Performance Considerations . . . 27
12.7 Testing and Debugging . . . 27
12.7.1 Development Workflow . . . 27
12.7.2 Common Issues . . . 27
12.8 Security Considerations . . . 27
12.8.1 Input Sanitization . . . 27
12.8.2 Credential Management . . . 27
12.8.3 Code Execution . . . 28

13 Conclusion **29**

14 Additional Resources **30**
14.1 Learning Ruby . . . 30
14.2 Plugin Development Tips . . . 30

15 Annex **31**
15.1 Adding a gem to Orchestrator 4.1.x . . . 31
15.1.1 Global Gem . . . 31
15.1.2 Plugin Gem . . . 31

Chapter 1

Introduction

This guide explains how to develop a plugin (action) for Aspera Orchestrator. Plugins allow you to extend Orchestrator's functionality by adding custom actions that can be used in workflows.

1.1 What is a Plugin?

A plugin (also called an ”action”) is a Ruby class that extends Orchestrator's capabilities.

Plugins can:

- Execute custom business logic
- Integrate with external systems (APIs, databases, file systems)
- Transform data and files
- Monitor events and trigger workflows
- Implement custom triggers that watch for specific conditions

1.2 How Plugins are Detected and Loaded

Orchestrator automatically discovers and loads plugins through the following mechanism:

1. Discovery: Plugins are stored in the `actions/` directory (configurable via config parameter `actionplugins_load_directory`)
2. Registration: The system scans the actions directory and registers each plugin in the database
3. Loading: When enabled, the system loads the Ruby class files and executes any migration files
4. Instantiation: The `Action` module provides the base interface that all plugins must implement
5. Execution: When a workflow step runs, Orchestrator instantiates the plugin class and calls its `execute` method

The plugin system uses `ActiveRecord` for persistence, allowing plugins to store configuration in database tables that are automatically created via migration files.

Chapter 2

Plugin Structure

An Aspera Orchestrator plugin consists of several files organized in a dedicated directory:

```
actions/my_plugins/
├── my_plugin.rb                    # Main plugin class (required)
├── metadata.yml                    # Metadata and version history (required)
├── edit.html.erb                   # Configuration UI (optional)
├── help.html.erb                   # Help documentation (optional)
├── MyPlugin.png                    # Plugin icon (optional, 48x48px recommended)
├── YYYYMMDDHHMMSS_*.rb            # Migration files (optional, for database schema)
├── my_plugin_controller.rb        # Controller for complex UI logic (optional)
├── my_plugin_helper.rb            # Helper methods for views (optional)
├── my_plugin_service.rb           # Service classes for business logic (optional)
└── additional_files/               # Additional support files (optional)
    ├── payloads/                   # XML/JSON templates
    ├── wsdl/                       # WSDL files for SOAP integrations
    └── sample_config/              # Configuration examples
```

2.1 Additional Ruby Files

Based on analysis of 205 plugins:

File Type	Usage	Purpose	Example
Controllers	~21 plugins	Complex UI interactions and AJAX operations	`akamai_transcoding_controller.rb`
Helpers	~8 plugins	View helper methods for formatting data and generating UI elements	`ateme_transcoding_helper.rb`
Services	~5 plugins	Encapsulate business logic and external API calls	`faspex5_package_monitor_authentication_service.rb`

2.2 File Naming Conventions

File Type	Naming Convention	Example
Main class file	Must match the class name in snake_case	`HelloWorld` → `hello_world.rb`
Migration files	Format `YYYYMMDDHHMMSS_description.rb` (timestamp ensures execution order)	`20231201120000_create_table.rb`

File Type	Naming Convention	Example
Icon file	Must match the class name	`HelloWorld.png`
View files	`edit.html.erb` for configuration, `help.html.erb` for documentation	`edit.html.erb`, `help.html.erb`

2.3 Directory Location

Plugins are installed in the directory specified by parameter `actionplugins_load_directory`, which defaults to `config/orchestrator/actions`. During development, plugins are typically placed in the `actions/` directory at the Orchestrator application root.

Chapter 3

Plugin Architecture Patterns

Plugins follow the following patterns and mechanisms:

3.1 Class Structure

All plugins must follow this structure:

```ruby
class MyPlugin < ActiveRecord::Base
  include Action

  # Constants for input/output variable names
  INPUT_PARAM = "input_param"
  OUTPUT_RESULT = "result"

  # Instance variables (inherited from Action module)
  # @inputs, @outputs, @status, @status_details, @percent_complete, @state_id
end
```

Key Points:

- Inherit from `ActiveRecord::Base` for database persistence
- Include the `Action` module which provides the plugin interface
- Define constants for all input/output variable names
- Use instance variables provided by the Action module

3.2 Core Methods

Every plugin must implement these essential methods defined in the `Action` module:

3.2.1 Required Methods

Method	Description
`self.version`	Returns plugin version as array `[major, minor, release]`
`inputs_spec`	Returns two `Hash`es: `required_inputs` and `optional_inputs`
`outputs_spec`	Returns `Hash` of output variables
`category`	Returns array defining plugin category
`execute`	Contains main execution logic

3.2.2 Optional Methods (for advanced features)

Method	Description
`validate_inputs`	Custom input validation
`pause`, `resume`, `cancel`, `rollback`	Lifecycle control
`timeout`	Returns timeout in seconds (0 = no timeout, nil = default)
`check_status`	Polls status for async operations
`synchronous_execution?`	Returns false for async execution
`multiple_execution?`	Returns true for trigger-type plugins
`remote_execution?`	Returns false to prevent remote execution
`recover`	Recovers from partial execution

3.3 Execution Patterns

Pattern	Configuration	Behavior	Examples
Synchronous Execution (default)	Default settings	`execute` method completes the work and returns final status: `[status, status_details, outputs]`	Simple file operations, API calls, data transformations
Asynchronous Execution	Override `synchronous_execution?` to return `false`	`execute` starts the operation and returns `STATUS_INPROGRESS`. Implement `check_status` to poll for completion	Transcoding jobs, large file transfers, batch processing
Trigger Pattern	Set `synchronous_execution?` to `false` and `multiple_execution?` to `true`	Use `check_status` to detect events and return `STATUS_COMPLETE` when triggered. Include `ModuleTriggerTools` module for trigger persistence	File monitoring, package arrival detection, webhook listeners

Chapter 4

Required Elements

4.1 Main Ruby Class

Each plugin must:

- Inherit from `ActiveRecord::Base`
- Include the `Action` module
- Define essential methods

4.2 Required Methods

4.2.1 `self.version`

Returns the plugin version by reading from `metadata.yml`.

```
def self.version
  return @@plugin_version_loaded if defined?(@@plugin_version_loaded)
  versions = SafeYaml.load_file(File.join(__dir__,'metadata.ym'))[:revision_history].map { |change|
↪   change[:version].split(".").map { |num| num.to_i } }
  @@plugin_version_loaded = versions.sort_by { |major,minor,revision,patch| [major, minor, revision,
  ↪  (patch.nil? ? 0 : patch)] }.last
  return @@plugin_version_loaded
end
```

4.2.2 `self.display_name` (Optional but Recommended)

Customizes the display name in the UI. Used by ~100 plugins.

```
def self.display_name
  return @@display_name if defined?(@@display_name)
  @@display_name = SafeYaml.load_file(File.join(__dir__,'metadata.ym'))[:display_name]
end
```

> **Note**
>
> If not defined, the system generates a name from the class name (e.g., `MyPlugin` → ”My plugin”).

4.2.3 `name` (Optional)

Provides a human-readable identifier for a configured instance.

Used by ~3 plugins for dynamic naming.

```
def name
  if attributes["name"].present?
```

```
    return attributes["name"]
  end
  # Generate dynamic name based on configuration
  return "#{workflow.name} sub-workflow" if workflow
  return "#{self.class.name}_#{id}"
end
```

Example: generates names based on the referenced workflow.

4.2.4 inputs_spec

Defines required and optional input parameters. This method is used for validation when saving a workflow.

```
def inputs_spec
  required_inputs = { "param1" => TYPE_STRING }
  optional_inputs = { "param2" => TYPE_INT }
  return required_inputs, optional_inputs
end
```

Available Input/Output Types:

Type	Description
TYPE_STRING	String value
TYPE_INT	Integer value
TYPE_ARRAY	Array value
TYPE_HASH	Hash/dictionary value
TYPE_FLAG	Boolean flag
TYPE_DATE	Date value
TYPE_OBJECT	Generic object
TYPE_PASSWORD	Password (masked in UI)
TYPE_ATTACHMENT	File attachment

4.2.5 outputs_spec

Defines output variables produced by the plugin. Sets the parameter name, type, and value to be output.

```
def outputs_spec
  return { "result" => TYPE_STRING }
end
```

4.2.6 category

Defines the plugin's category in the UI. Available categories:

```
def category
  return [CATEGORY_FILEOPERATIONS]
end
```

Available Categories:

Category	Description
CATEGORY_ASSET	
CATEGORY_FILEOPERATIONS	File Operations
CATEGORY_FILETRANSFER	File Transfer
CATEGORY_FILETRANSFORMATIONS	File Transformations
CATEGORY_INTEGRATION	Integration
CATEGORY_LIST	
CATEGORY_OTHER	Other Utilities
CATEGORY_QUALITY	
CATEGORY_QUALITY_CONTROL	Quality Control

Category	Description
`CATEGORY_RESOURCE`	
`CATEGORY_SCHEDULING`	Scheduling
`CATEGORY_SYSTEM`	System
`CATEGORY_TRANSCODING`	Transcoding
`CATEGORY_TRIGGERS`	Triggers
`CATEGORY_USERINTERACTIONS`	User Interactions
`CATEGORY_VIRUS`	

4.2.7 `execute`

Contains the main execution logic. This method must return three values:

```
def execute
  @outputs = {}
  # Execution logic here
  @outputs["result"] = "value"
  return STATUS_COMPLETE, "Execution successful", @outputs
end
```

Return Values:

1. @status - Plugin execution status (see Status Constants below)
2. @status_details - String describing the current state (e.g., ”Processing complete”, ”Error occurred”)
3. @outputs - Hash containing output variables defined in `outputs_spec`

Instance Variables:

- `@inputs` - Hash of input values provided by the workflow engine
- `@outputs` - Hash of output values to return to the workflow engine

Status Constants:

Status	Description
`STATUS_COMPLETE`	Execution completed successfully
`STATUS_INPROGRESS`	Execution in progress (for asynchronous plugins)
`STATUS_FAILED`	Execution failed
`STATUS_ERROR`	Error occurred during execution
`STATUS_PAUSED`	Execution paused
`STATUS_PAUSING`	Execution is pausing
`STATUS_CANCELING`	Execution is being canceled
`STATUS_RESUMING`	Execution is resuming
`STATUS_ROLLINGBACK`	Execution is rolling back
`STATUS_INACTIVE`	Plugin is inactive
`STATUS_ACTIVATING`	Plugin is activating
`STATUS_OBSOLETE`	Plugin is obsolete
`STATUS_UNDEFINED`	Status could not be determined
`STATUS_TRUE`	Boolean true status
`STATUS_FALSE`	Boolean false status

4.3 `metadata.yml` File

The `metadata.yml` file contains plugin metadata:

```
---
:revision_history:
  - :author: "Your Name"
    :change_description: Initial release
```

```
    :version: 1.0.0
    :date: "2024-01-01"
    :minimum_server_version: 4.1.0

:display_name: My Plugin
:category: File Operations
:help: "Description of what the plugin does"
:plugin_name: MyPlugin
:description: Short description of the plugin
```

Chapter 5

API Reference

This section provides reference information for constants and types used in plugin development.

5.1 Data Types

Available data types for inputs/outputs:

Type	Description
`TYPE_STRING`	String value
`TYPE_INT`	Integer number
`TYPE_FLOAT`	Float/Decimal number
`TYPE_FLAG`	Boolean (true/false)
`TYPE_HASH`	Hash/Dictionary structure
`TYPE_ARRAY`	Array/List structure

5.2 Execution Status

Possible statuses returned by `execute`:

Status	Description
`STATUS_COMPLETE`	Successful execution
`STATUS_FAILED`	Execution failed
`STATUS_ERROR`	System error
`STATUS_INPROGRESS`	In progress (for asynchronous execution)
`STATUS_PAUSED`	Paused

Chapter 6

Simple Example: ”Hello World” Plugin

Here's a minimal plugin example that displays a personalized message:

6.1 Directory Structure

```
actions/hello_world_simple/
├── hello_world_simple.rb
└── metadata.yml
```

6.2 `hello_world_simple.rb`

```ruby
require 'yaml'

class HelloWorldSimple < ActiveRecord::Base
  include Action

  # Output variable name
  MESSAGE_OUTPUT = "message"

  # Input variable name
  NAME_INPUT = "name"

  # Version method - reads from metadata.yml
  def self.version
    return @@plugin_version_loaded if defined?(@@plugin_version_loaded)
    versions = SafeYaml.load_file("#{File.dirname(__FILE__)}/metadata.yml")[:revision_history].map {
↪  |change| change[:version].split(".").map { |num| num.to_i } }
    @@plugin_version_loaded = versions.sort_by { |major,minor,revision,patch| [major, minor,
     ↪  revision, (patch.nil? ? 0 : patch)] }.last
    return @@plugin_version_loaded
  end

  # Define inputs: one required parameter "name"
  def inputs_spec
    required_inputs = { NAME_INPUT => TYPE_STRING }
    optional_inputs = {}
    return required_inputs, optional_inputs
  end

  # Define outputs: one output "message"
  def outputs_spec
    return { MESSAGE_OUTPUT => TYPE_STRING }
  end

  # Plugin category
  def category
```

```
    return [CATEGORY_OTHER]
  end

  # Main execution method
  def execute
    begin
      # Initialize outputs
      @outputs = {}

      # Get the name from inputs
      name = @inputs[NAME_INPUT]

      # Create the message
      message = "Hello, #{name}!"

      # Set the output
      @outputs[MESSAGE_OUTPUT] = message

      # Return success status
      return STATUS_COMPLETE, "Message created successfully", @outputs

    rescue Exception => e
      # Handle errors
      return STATUS_ERROR, "Error: #{e.message}", {}
    end
  end
end
```

6.3 metadata.yml

```
---
:revision_history:
  - :author: "Developer"
    :change_description: Initial release
    :version: 1.0.0
    :date: "2024-01-01"
    :minimum_server_version: 4.1.0

:display_name: Hello World Simple
:category: Other Utilities
:help: "This plugin creates a personalized greeting message"
:plugin_name: HelloWorldSimple
:description: A simple plugin that demonstrates the basic structure of an Orchestrator action
```

6.4 How It Works

1. Input: The plugin receives a `name` parameter
2. Processing: It creates a greeting message: ”Hello, [name]!”
3. Output: Returns the message in the `message` output variable
4. Status: Returns `STATUS_COMPLETE` on success

Chapter 7

Testing Your Plugin

7.1 Development vs Production Paths

- Development path: `/opt/aspera/orchestrator/actions` - Place plugins here during development
- Production path: `/opt/aspera/var/config/orchestrator/actions` - Plugins are loaded from here (configurable via `actionplugins_load_directory`)
- Database tables are updated when plugins are moved from development to production path

7.2 Testing Steps

1. Place your plugin directory in the development `actions/` directory
2. Restart Orchestrator to load the plugin
3. The plugin will appear in the action list under the specified category
4. Create a workflow step using your plugin
5. Provide the required inputs
6. Execute and verify the outputs

7.3 Important Notes

- No empty classes: Orchestrator will attempt to process empty classes and throw errors if they are present
- JavaScript in views: JavaScript code should be added directly to the `edit.html.erb` file, not in separate files
- Database columns: All plugins inherit standard columns from the `Action` base class (name, comments, etc.)

Chapter 8

Advanced Features

8.1 Database Migrations

Plugins can create and modify database tables using migration files. This is essential for storing plugin configuration and state.

Migration File Structure:

```ruby
# File: 20240101120000_create_my_plugins.rb
class CreateMyPlugins < ActiveRecord::Migration[7.2]
  def change
    create_table :action_my_plugins do |t|
      t.string :name
      t.text :comments
      t.string :my_config_field
      t.timestamps
    end
  end
end
```

Key Points:

- Migration files must be prefixed with timestamp: `YYYYMMDDHHMMSS_description.rb`
- File must start with underscore: `_create_my_plugins.rb`
- File name must be plural and match the folder name (tableize is used to validate)
- Table name must follow pattern: `action_#{plugin_name.tableize}` (e.g., `action_my_plugins`)
- Migrations are executed automatically when plugin is loaded
- Use standard `Rails` migration methods: `create_table`, `add_column`, `change_column`, etc.
- Multiple migration files can exist in a plugin directory

Versioning Migrations: When updating a plugin, create new migration files with later timestamps:

```ruby
# File: 20240201120000_my_plugins_add_new_field.rb
class MyPluginsAddNewField < ActiveRecord::Migration[7.2]
  def change
    add_column :action_my_plugins, :new_field, :string
  end
end
```

8.2 Asynchronous Execution

For long-running operations that shouldn't block the workflow engine:

```ruby
def synchronous_execution?
  return false  # Indicates asynchronous execution
end

def execute
  # Start the operation (e.g., submit job to external system)
```

```
  @job_id = submit_job_to_external_system(@inputs)
  @status = STATUS_INPROGRESS
  return @status, "Job submitted: #{@job_id}", @outputs
end

def check_status
  # Poll external system for job status
  job_status = query_external_system(@job_id)

  if job_status == "complete"
    @outputs["result"] = get_job_result(@job_id)
    return STATUS_COMPLETE, "Job finished", @outputs
  elsif job_status == "failed"
    return STATUS_FAILED, "Job failed", @outputs
  else
    # Return polling interval (in seconds) as third parameter
    return STATUS_INPROGRESS, "Still processing", 30
  end
end
```

Important Notes:

- `synchronous_execution?` should return `false` to indicate asynchronous execution
- This prevents the action from blocking the Slave Queue Thread
- The third return value from `check_status` specifies the polling interval in seconds
- Only use for triggers and long-running processes - synchronous execution should be `true` for most plugins

8.3 Trigger Plugins

Triggers watch for events and spawn new workflow instances. They use the trigger pattern:

```
class MyTrigger < ActiveRecord::Base
  include Action
  include ModuleTriggerTools  # Provides trigger persistence methods

  def synchronous_execution?
    return false  # Triggers are always async
  end

  def multiple_execution?
    return (keep_ongoing == true) && (@is_canceled != true)
  end

  def execute
    @outputs = {}
    return nil, "Initializing trigger", nil
  end

  def check_status
    # Check for trigger condition
    if event_detected?
      @outputs["event_data"] = get_event_data

      # Persist trigger to avoid re-triggering
      register_triggers([event_identifier])
      persist_triggers(@current_triggers)

      return STATUS_COMPLETE, "Event detected", @outputs
    else
      return nil, "Waiting for event", polling_frequency
    end
  end
end
```

Trigger Persistence:

- Use `@current_triggers` to get previously triggered events
- Use `register_triggers([event_id])` to mark new events
- Use `persist_triggers(@current_triggers)` to save state
- Include `ModuleTriggerTools` module for trigger persistence methods

Important Notes:

- `synchronous_execution?` must return `false` for triggers
- `multiple_execution?` should return `true` to allow multiple workflow instances
- Triggers sit idle until an event occurs, so they shouldn't block the queue
- Use trigger persistence to avoid re-triggering on the same event

8.4 Lifecycle Control (Pause/Resume/Cancel/Rollback)

Plugins can support workflow control operations:

```
def pausable?
  return true  # Indicates pause is supported
end

def pause
  # Pause the operation
  @is_paused = true
  report_progress(STATUS_PAUSED, "Operation paused")
  return true
end

def resumable?
  return true
end

def resume
  # Resume the operation
  @is_paused = false
  return true
end

def cancelable?
  return true
end

def cancel
  # Cancel the operation
  cleanup_resources
  @status = STATUS_FAILED
  @status_details = "Operation canceled"
  return true
end

def rollbackable?
  return true
end

def rollback
  # Undo changes made by this plugin
  undo_changes
  return true
end
```

Note: By default, Orchestrator checks if these methods are overridden using `is_overloaded?`. See for a complete implementation.

8.5 Input Validation

Custom validation beyond type checking:

```
def validate_inputs(inputs_hash = @inputs)
  # First perform default validation
  return false unless default_inputs_validation(inputs_hash)

  # Custom validation logic
  if inputs_hash["number"].present?
    return false if inputs_hash["number"].to_i < 0
  end

  if inputs_hash["email"].present?
    return false unless
     ↪  inputs_hash["email"].match?(/\A[\w+\-.]+@[a-z\d\-]+(\.[a-z\d\-]+)*\.[a-z]+\z/i)
  end

  return true
end
```

8.6 Helper Methods

`ActionTools` (included by `Action`) provides `default_get(field_name)`: the run time value of a template attribute. It takes the first of:

1. The input that replaces the attribute when blank (see `implied_field_spec`: constant `VAR_<FIELD>`, else the attribute name with its first letter in upper case).
2. The attribute value, if not blank.
3. The constant `DEFAULT_<FIELD>`.

A `string` or `text` value is then expanded with `Payload.expand`: placeholders are replaced by input values. If the attribute name ends with `_<type>` (for example `delay_int`), the value is converted to that type.

```
DEFAULT_POLLING_FREQUENCY = 5

def polling_frequency_get
  return default_get(:polling_frequency)
end
```

8.7 Dynamic Input/Output Specifications

`ActionTools#default_inputs_spec(fields, required_inputs, optional_inputs)` derives inputs from template attributes (default: all attributes, into `@required_inputs` and `@optional_inputs`), and returns the two maps:

- A text attribute left blank becomes an input (`implied_field_spec`), optional if the constant `DEFAULT_<FIELD>` exists.
- A text attribute that is set gives one input per placeholder (`Payload.variables`): `<%= name %>`, arguments of `<%= function(a, b) %>`, `var(name)` in an expression. The input is a `string`, or the type of a suffix `_<type>` of its name, required unless the constant `DEFAULT_<NAME>` exists. No input is created for a name that is a method of the template with a value (`name`, a set attribute).
- A non-text attribute left null becomes an input of the corresponding type.

```
def inputs_spec
  required_inputs = {}
  optional_inputs = {"timeout" => TYPE_INT}
  # command: input Command if blank, else one input per placeholder
  default_inputs_spec([:command], required_inputs, optional_inputs)
  return required_inputs, optional_inputs
end
```

An attribute handled by your own code (for example `required_inputs[VAR_SERVER] = TYPE_STRING if server.blank?`) gets no input for its placeholders: call `default_inputs_spec` for it, or scan it with `Payload.`

`variables` (see Using Payload Variables). The documentation generator marks an attribute `placeholders: true` in `plugins.yml` only when `<%= variable %>` in its value gives an input `variable`. See Placeholders in the Workflow Authoring Guide for the syntax as seen by workflow authors.

Chapter 9

Plugin Patterns Usage Statistics

Based on analysis of 205 plugins in the Orchestrator codebase:

9.1 Common Patterns

Pattern	Usage	Description
`SafeYaml` for version	203/205 (99%)	Standard pattern for reading `metadata.yml`
`self.display_name`	100/205 (49%)	Custom display names in UI
`MultiType` utilities	88/205 (43%)	Type conversion and validation
Asynchronous execution	80/205 (39%)	Long-running operations
`dependencies` method	74/205 (36%)	External entity dependencies
`Payload` variables	58/205 (28%)	Dynamic input extraction
`cancel` support	58/205 (28%)	Cancellation capability
`timeout` definition	54/205 (26%)	Custom timeout logic
`validate_inputs`	40/205 (20%)	Custom validation
Trigger type	37/205 (18%)	Event-driven workflows
`pause`/`resume`	34/205 (17%)	Lifecycle control
`Gemfile` dependencies	34/205 (17%)	External Ruby gems
`recover` method	29/205 (14%)	Partial execution recovery
`remote_execution?` = false	25/205 (12%)	Local-only execution
Controller files	21/205 (10%)	Complex UI logic
Helper files	8/205 (4%)	View helpers
Service files	5/205 (2%)	Business logic separation
`rollback` support	2/205 (1%)	Undo capability

9.2 Key Takeaways

1. Version Management: Nearly all plugins use the standard `SafeYaml` pattern for version reading
2. Display Names: Half of plugins customize their display name for better UX
3. Type Handling: `MultiType` is widely used for robust type conversion
4. Async Operations: 39% of plugins use asynchronous execution for long-running tasks
5. Dependencies: Over a third declare dependencies on other entities
6. Advanced Features: Pause/resume, cancel, and recover are used selectively where needed

Chapter 10

Best Practices

1. Error Handling: Always wrap execution logic in try/catch blocks
2. Logging: Use `Rails.logger` for debugging and error messages
3. Constants: Define constants for variable names and default values
4. Documentation: Provide clear help text and parameter descriptions
5. Versioning: Update `metadata.yml` with each change
6. Testing: Test with various input combinations
7. Cleanup: Clean up temporary files and resources
8. Progress Reporting: Use `report_progress` for long operations

Chapter 11

Advanced Usage Patterns

11.1 Using Payload Variables

`Payload.variables(text)` returns the input names used by the placeholders of `text`:

Placeholder	Names returned
`<%= name %>`	`name`
`<%= function(a, b) %>`	`a`, `b` (arguments are input names only)
`<%= var(name).upcase %>`	`name` (`var` in an expression)
`<%= name.upcase %>`, `<% ... %>`	none

`Payload.expand(text, @inputs, self)` replaces them with input values:

- `<%= name %>`: the input value, an empty string if missing (or the value of method `name` of `self` if there is no such input), an array in Ruby notation.
- `<%= function(a, b) %>`: `function` is a method of `self` (the template), of `Payload` (`xml`: a hash as XML elements), or a method with its class (`File.basename`).
- If `<%=` remains after that, the text is evaluated as ERB, where a name is the value of the input with that name (`var(name)` returns its argument). `<% ... %>` tags are evaluated only in that case.
- `<%= var(name) %>` alone raises an error: `var` is a method of `Payload` class only.

Scan with `Payload.variables` when `default_inputs_spec` does not fit, for example to give all inputs one type:

```
def inputs_spec
  required_inputs = {}
  # Extract variables from a payload string
  Payload.variables(command).each do |var_name|
    required_inputs[var_name] = TYPE_STRING
  end
  return required_inputs, {}
end

def execute
  # Expand payload with actual values (default_get(:command) does the same)
  expanded_command = Payload.expand(command, @inputs, self)
  # ... use expanded_command ...
end
```

11.2 Remote Node Integration

```
def dependencies
  return [{
    entity: "RemoteNode",
    id_key: "name",
```

```
    id_value: remote_node,
    dependent_field: "remote_node"
  }]
end
```

11.3 Multi-Type Conversion

```
# Convert string to typed value
value = MultiType.convert_value(string_value, TYPE_INT)

# Convert value to string
string = MultiType.convert_to_string(value)

# Convert hash representation
hash = MultiType.convert_value_hash(string_hash)
```

Chapter 12

Plugin Development Alternatives and Possibilities

12.1 Plugin Types and Use Cases

Orchestrator supports several types of plugins, each suited for different scenarios:

Plugin Type	Purpose	Execution	Examples	Best For	Key Features
Standard Action Plugins	Execute specific tasks within a workflow	Synchronous or asynchronous	File operations, API calls, data transformations	One-time operations with clear inputs and outputs	-
Trigger Plugins	Monitor for events and spawn new workflow instances	Always asynchronous with `multiple_execution?` = true	File system monitors, message queue listeners	Event-driven workflows, continuous monitoring	Include `ModuleTriggerTools` for trigger persistence
Integration Plugins	Connect to external systems and services	Varies	SOAP/REST APIs, databases, cloud services	System integration, data exchange	Use `Gemfile` for dependencies, store WSDL files, use payload templates
Custom Trigger with Code Evaluation	Maximum flexibility for custom logic	User-defined	Custom Ruby code execution	Power users who need complete control	Allows custom Ruby code for inputs, outputs, validation, and execution. Warning: Security implications

12.2 Configuration UI Options

Plugins can provide custom configuration interfaces:

12.2.1 Standard Form Fields

Use Rails form helpers in :

```
<%= form_with model: @my_plugin, url: action_update_path(action_type: "my_plugin", id: @my_plugin.id)
↪  do |f| %>
  <li>
    <%= f.label :name %>
    <%= f.text_field :name %>
  </li>
  <li>
```

```
    <%= f.label :config_value %>
    <%= f.text_area :config_value %>
  </li>
<% end %>
```

12.2.2 Dynamic Configuration

Display runtime-calculated values:

```
<% if @view == 'show' %>
  <li>
    <%= f.label 'Required Variables' %>
    <%= h Payload.variables(@my_plugin.command).join(', ') %>
  </li>
<% end %>
```

12.2.3 Help Documentation

Provide detailed help in `help.html.erb` with HTML formatting, examples, and usage instructions.

12.3 External Dependencies Management

12.3.1 Using Gemfile

For plugins requiring external Ruby gems:

```
# File: actions/my_plugin/Gemfile
source 'https://rubygems.org'

gem 'aws-sdk-sqs', '~> 1.0'
gem 'rest-client', '~> 2.1'
```

Dependencies are automatically installed when the plugin is loaded.

12.3.2 System Dependencies

For external tools (e.g., ffmpeg, exiftool):

- Document in `metadata.yml` under `:external_dependencies`
- Check availability in plugin code
- Provide clear error messages if missing

12.4 Plugin Versioning and Upgrades

12.4.1 Version Management

- Versions are defined in key `:revision_history` in `metadata.yml`
- Format: `major.minor.release` (e.g., ”1.2.3”)
- Each version entry includes: author, change_description, version, date
- Optionally specify `minimum_server_version` and `maximum_server_version`

12.4.2 Upgrade Process

When upgrading a plugin:

1. Create new migration files with later timestamps
2. Update version in `metadata.yml`
3. The system automatically detects version changes
4. Migrations are executed in order
5. Old plugin instances are preserved for rollback

12.4.3 Deprecation

Mark plugins as deprecated:

```
:revision_history:
  - :deprecation_orch_version: "4.2.0"
```

12.5 Plugin Distribution

12.5.1 Installation

Place in `actions/` directory and reload

12.6 Performance Considerations

Efficient Execution

- Use asynchronous execution for long-running operations
- Implement proper timeout values via `timeout` method
- Return appropriate polling intervals in `check_status`

Resource Management

- Clean up temporary files and connections
- Use `remote_execution?` = true to allow execution on remote nodes
- Implement proper error handling and recovery

12.7 Testing and Debugging

12.7.1 Development Workflow

1. Place plugin in `actions/` directory
2. Reload plugins via UI or API
3. Test in workflow designer
4. Check logs: `Rails.logger.info`, `Rails.logger.error`
5. Use Action Tester for isolated testing

12.7.2 Common Issues

- Plugin not appearing: Check `metadata.yml` syntax, verify file naming
- Migration errors: Ensure table name follows `action_*` pattern
- Version conflicts: Verify version comparison logic
- Loading failures: Check Ruby syntax, verify all required methods are implemented

12.8 Security Considerations

12.8.1 Input Sanitization

- Always validate and sanitize user inputs
- Use parameterized queries for database operations
- Escape shell commands properly

12.8.2 Credential Management

- Never hardcode credentials
- Use Orchestrator's credential management
- Support environment variables for sensitive data

12.8.3 Code Execution

- Be cautious with `eval`
- Validate file paths to prevent directory traversal
- Limit file system access to designated directories

Chapter 13

Conclusion

This comprehensive guide provides the foundation for developing Aspera Orchestrator plugins. Key takeaways:

1. Start Simple: Begin with the basic example and gradually add complexity
2. Follow Patterns: Study existing plugins in `actions/` for proven patterns
3. Use the Framework: Leverage the `Action` module and helper methods
4. Test Thoroughly: Use the Action Tester and test with various input combinations
5. Document Well: Provide clear `metadata.yml`, help files, and inline comments
6. Consider Performance: Use async execution and proper resource management
7. Plan for Upgrades: Use migrations for schema changes and version properly

For more examples and advanced techniques, explore the existing plugins in the `actions/` directory, particularly:

- Basic plugin with pause/resume/cancel
- Trigger pattern with persistence
- Dynamic code evaluation
- External gem dependencies

Chapter 14

Additional Resources

14.1 Learning Ruby

For developers new to Ruby, these resources are recommended:

- Ruby Tutorial - TutorialsPoint - Comprehensive Ruby tutorial
- Codecademy Ruby Track - Interactive Ruby learning
- Try Ruby - Browser-based Ruby introduction
- Ruby Monk - Interactive Ruby tutorials

14.2 Plugin Development Tips

1. Folder Naming: Plugin folder must be plural (e.g., `cerify_file_verifications`)
2. Migration Naming: Must start with underscore, be plural, and match folder name
3. Controller Usage: Add controllers for plugins requiring server-supplied data for the action template
   - Name: `{plugin_name}_controller.rb` (e.g., `cerify_file_verifications_controller.rb`)
4. View Files: All forms push data to the back-end from fields present in `edit.html.erb`
5. Engine Variables: The engine only sees `@inputs` and `@outputs` variables in the plugin
6. Return Format: All plugins must return `[@status, @status_details, @outputs]`
7. Empty Classes: No empty classes are allowed - Orchestrator will attempt to process them and throw errors

Chapter 15

Annex

15.1 Adding a gem to Orchestrator 4.1.x

15.1.1 Global Gem

Edit the file: `/opt/aspera/orchestrator/Gemfile`

Example, add:

```
gem 'prawn'
gem 'prawn-table'
```

Then do:

```
cd /opt/aspera/orchestrator/
bundle install
orchestrator restart
```

15.1.2 Plugin Gem

Each plugin can have its own dependencies in:

```
/opt/aspera/orchestrator/actions/<PLUGIN_NAME>/Gemfile
```

Install dependencies like for global gems.

End of document